NutriVision – Project Planning & Management

Project Documentation – Section 1

1. Project Proposal

Project Overview

NutriVision: AI-Driven Dietary Health Assistant is a Flutter-based mobile application designed to help users manage their dietary habits through AI-powered nutritional analysis and personalized recommendations. Users can record meals using natural-language descriptions and receive nutritional advice, healthier alternatives, and customized recipe suggestions based on dietary preferences and restrictions.

Project Objectives

- Develop a cross-platform mobile application using Flutter.
- Allow users to record and manage meals.
- Analyze meal descriptions using Artificial Intelligence.
- Generate personalized nutritional recommendations.
- Provide healthier recipe and snack alternatives.
- Support dietary restrictions and user preferences.
- Synchronize meal data across devices using Firebase Cloud Firestore.
- Implement secure API communication using Dio and Interceptors.
- Provide responsive user interfaces for different screen sizes.
- Validate core logic and UI behavior through unit and widget testing.

Project Scope

Included:

- Meal logging and meal history.
- AI nutritional analysis.
- Personalized recipe generation.
- Healthy food alternative suggestions.
- Dietary restriction support.
- Nutritional charts and dashboard.
- Firebase Cloud Firestore synchronization.
- Dio-based API communication.
- Responsive UI.
- Unit and widget testing.

2. Project Plan

Week	Activities	Deliverable
1	Requirements Gathering	Requirements Document
2	Literature Review and Research	Literature Review
3	UI/UX and Design Thinking	Wireframes and Personas
4	Application Architecture	System Architecture
5	Flutter UI Development	Initial Application UI
6	AI/API Integration	Working AI Features
7	Firebase Integration	Cloud Data Synchronization
8	Meal History and Charts	Completed Dashboard
9	Testing and Debugging	Test Results
10	Documentation and Presentation	Final Report and Presentation

3. Task Assignment & Roles

Based on the team members' expertise and skill levels, responsibilities are structured as follows:

Team Member	Module Name & Domain	Flutter UI / State Responsibilities	Backend, AI & Business Logic Responsibilities
Mahmoud *(Team Leader)*	**Core Networking & External API Lookup Module**	• Builds NutritionalLookupScreen with real-time search inputs and live suggestion lists. • Implements network state indicators (Loading, Error, Offline Banner).	• Overall sprint administration and GitHub repository workflow management.

Team Member	Module Name & Domain	Flutter UI / State Responsibilities	Backend, AI & Business Logic Responsibilities
			• Constructs Dio Service Layer with custom Interceptors for API key encryption, token refresh, and request logging. • Parses external JSON payloads into strongly-typed Dart models.
Ahmed	**AI Prompt Engine & Recipe Generation Module**	• Builds AIRecipeGeneratorScreen featuring dynamic filter chips, allergen toggles, and recipe detail sheets. • Manages local UI form states and dynamic prompt preview cards.	• Authors and maintains all official DEPI documentation files. • Engineers system and user prompt templates enforcing strict JSON output

Team Member	Module Name & Domain	Flutter UI / State Responsibilities	Backend, AI & Business Logic Responsibilities
			schemas for recipes. ● Integrates LLM REST endpoints and handles error recovery for malformed AI responses.
Kamel	**Real-Time Firestore Sync & Meal History Module**	● Builds MealHistoryScreen with interactive timeline logs, swipe-to-delete, and date filtering. ● Implements optimistic UI updates for instant UI feedback.	● Configures Firebase Authentication and Cloud Firestore security rules. ● Implements reactive Firestore StreamBuilder bindings in Flutter for multi-device sync. ● Handles local cache persistence for

Team Member	Module Name & Domain	Flutter UI / State Responsibilities	Backend, AI & Business Logic Responsibilities
			offline meal entry.
Osama	**AI Snack Search & Discovery Module**	• Builds SnackDiscoveryScreen featuring side-by-side visual comparison cards (Unhealthy Craving vs. Healthy AI Alternative). • Designs interactive bookmarking and alternative swap UI components.	• Engineers search & discovery prompt workflows analyzing user craving inputs against macro goals. • Implements local storage / cloud sync for saved healthy favorite alternatives. • Handles ingredient substitution logic calculations.
Bassam	**Dashboard Telemetry, Visual Charts & QA Module**	• Builds NutritionalDashboardScreen utilizing custom GridView layouts and layered Stack widgets. • Renders daily calorie progress rings and macro breakdown charts.	• Programs daily total calorie/macro summation algorithms and

Team Member	Module Name & Domain	Flutter UI / State Responsibilities	Backend, AI & Business Logic Responsibilities
			target progress logic. • Leads Quality Assurance by writing automated Unit tests for calculations and Widget tests for UI responsiveness. • Enforces design system consistency, dark/light theme tokens, and responsive scaling.

4. Risk Assessment & Mitigation Plan

Risk	Probability	Impact	Mitigation
Inaccurate AI nutritional output	Medium	High	Use controlled prompts, validation and clear informational disclaimers.

External API unavailable	Medium	High	Implement timeout handling, error states and fallback messages.
Internet connection failure	Medium	Medium	Cache appropriate local data and handle offline states.
Firebase synchronization failure	Low/Medium	High	Use retry/error handling and robust Firestore rules.
Data privacy/security issues	Medium	High	Use HTTPS, secure credentials and restricted database access.
UI performance issues	Medium	Medium	Optimize widgets and avoid unnecessary rebuilds.
Testing delays	Medium	Medium	Test incrementally throughout development.
Scope expansion	High	Medium	Maintain a clearly defined project scope and prioritize core features.

5. KPIs (Key Performance Indicators)

KPI	Target / Measurement
Application response time	Target approximately ≤ 3 seconds for normal non-AI UI/API operations.
API error handling	All identified API failure scenarios should produce controlled user feedback.
Firebase synchronization	Successful synchronization under normal connectivity.

Test pass rate	Target ≥ 90% for implemented automated tests.
Crash/error rate	Minimize crashes during functional and usability testing.
UI responsiveness	Core screens operate correctly across supported screen sizes.
User adoption	Measure during usability evaluation.
Meal logging completion	Measure the percentage of attempted meal entries successfully saved.